北京師範大學
BEIJING NORMAL UNIVERSITY

北京師範大學
人工智能学院

《计算机图形学》课程实验

实验二：变换与坐标系统

参考流程与答疑

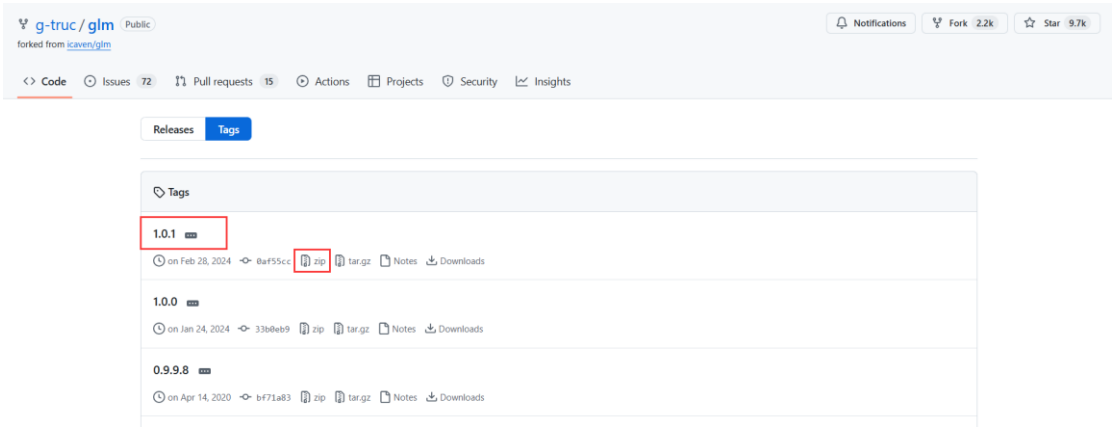
授课教师：张鸿文

助教：李玉慧

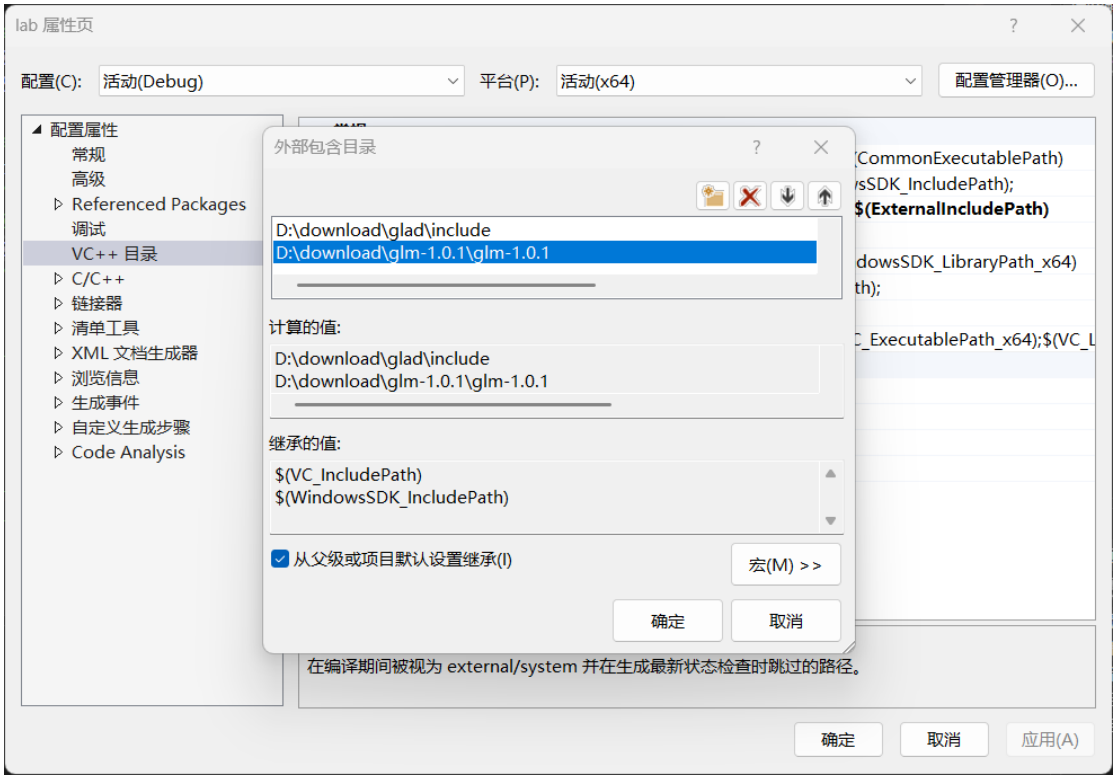
2025. 03

计算机图形学实验二——图形变换：变换与坐标系统

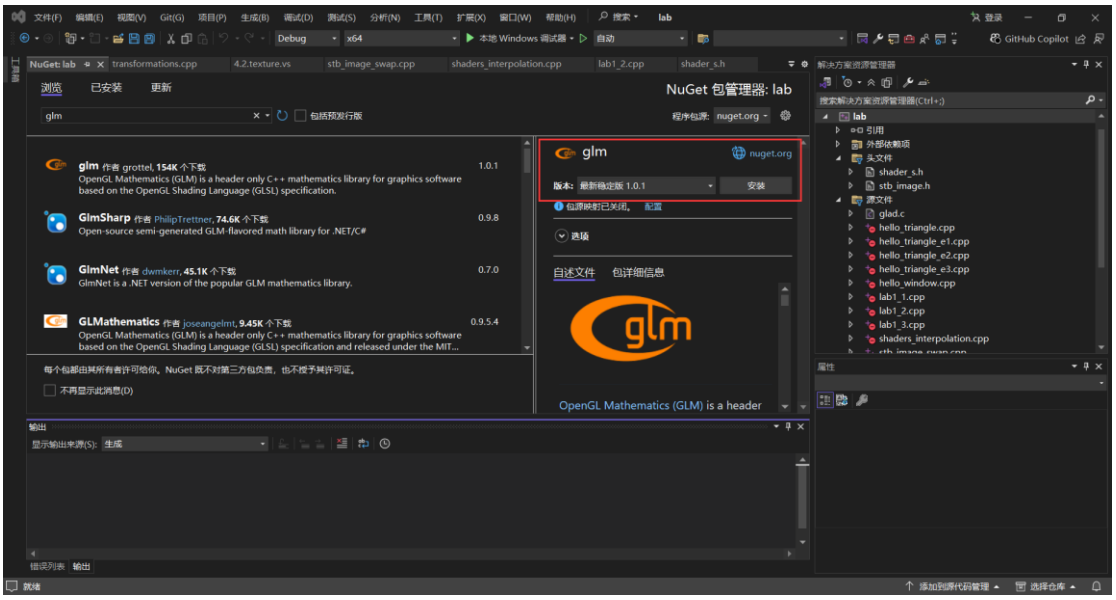
1. 安装 glm 库（教程 0.9.8 版，最新 1.0.1，下文用最新版），流程与 glad 相同
   https://glm.g-truc.net/0.9.8/index.html
   https://github.com/g-truc/glm/tags



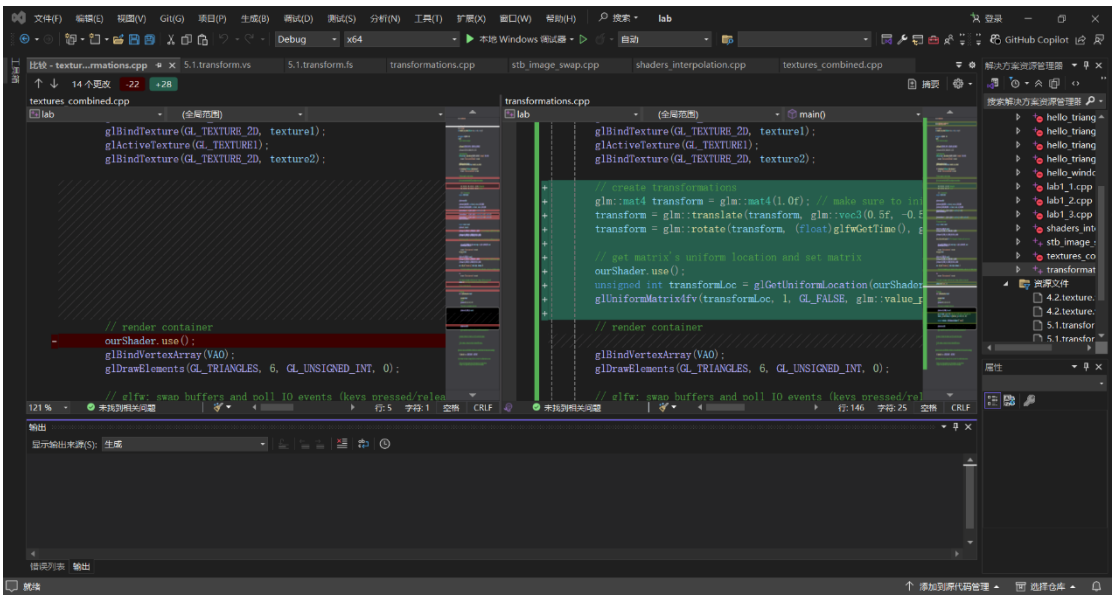
根目录即为 glm-1.0.1



除了从官网下载，也可以使用 NuGeT 程序包管理器下载 glm



2. **阅读变换一节，运行示例程序“transformations.cpp”**
   https://learnopengl.com/code_viewer_gh.php?code=src/1.getting_started/5.1.transformations/transformations.cpp
   将”transformations.cpp”与“textures_combined.cpp”对比，主要区别有：增加了 glm 变换相关函数，着色器的修改。



3. **在实验一的基础上，为带纹理的立方体增加旋转变换。**
   着色器代码：

```
const char* vertexShaderSource = "#version 330 core\n"
"layout (location = 0) in vec3 aPos;\n"
"layout (location = 1) in vec3 aColor;\n"
"layout (location = 2) in vec2 aTexCoord;\n"
"out vec3 ourColor;\n"
```

```
"out vec2 TexCoord;\n"
"uniform mat4 transform;\n"
"void main()\n"
"{\n"
"    gl_Position = transform * vec4(aPos, 1.0);\n"
"    ourColor = aColor;\n"
"    TexCoord = vec2(aTexCoord.x, aTexCoord.y);\n"
"}\0";
```

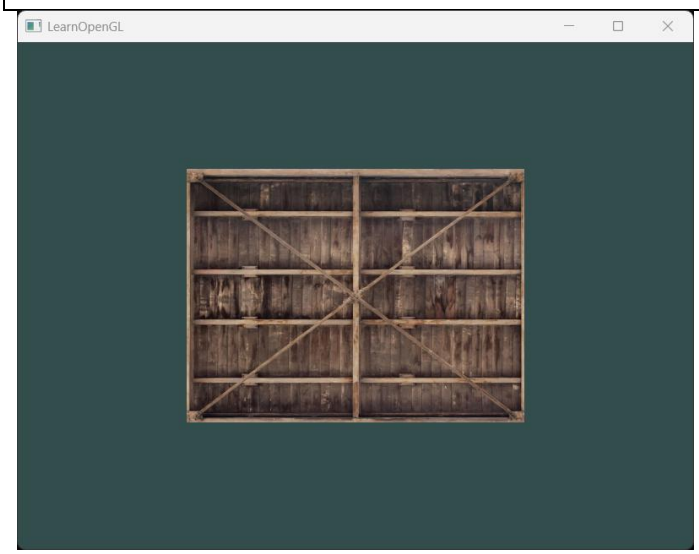
参考立方体形状，左上角向前。注意，在图像坐标系中，y 轴以向上为正，x 轴以向右为正，z 轴以远离屏幕为正（左手坐标系）。

```
    float vertices[] = {
         0.5f,  0.5f, 0.0f, 0.0f, 0.0f, 1.0f, 1.0f, 1.0f,   // top right
         0.5f, -0.5f, 0.0f, 0.0f, 1.0f, 0.0f, 1.0f, 0.0f,// bottom right
        -0.5f, -0.5f, 0.0f, 1.0f, 0.0f, 0.0f, 0.0f, 0.0f,// bottom left
        -0.5f,  0.5f, -1.0f, 0.0f, 0.0f, 0.0f, 0.0f, 1.0f // top left -> front
    };

    unsigned int indices[] = {  // note that we start from 0!
        0, 1, 2,  // first Triangle
        2, 3, 0,  // first Triangle
        3, 0, 1,   // second Triangle
        1, 2, 3,   // second Triangle
    };
```

矩阵传递过程参考示例代码自行修改，并结合教程理解其原理。修改变换矩阵，更改旋转轴，依次显示三个不同视角的视图。

正视图（不旋转）：

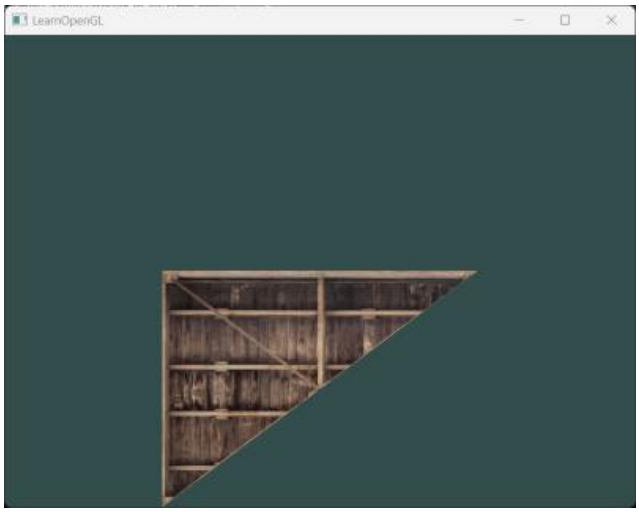
```
glm::mat4 model= glm::mat4(1.0f);
```



俯视图（物体绕 x 轴逆时针旋转 90 度）如下。glm 的 rotate 基于左手坐标系，在左手坐标系中，从旋转轴的上方看，顺时针旋转则被视为正方向。

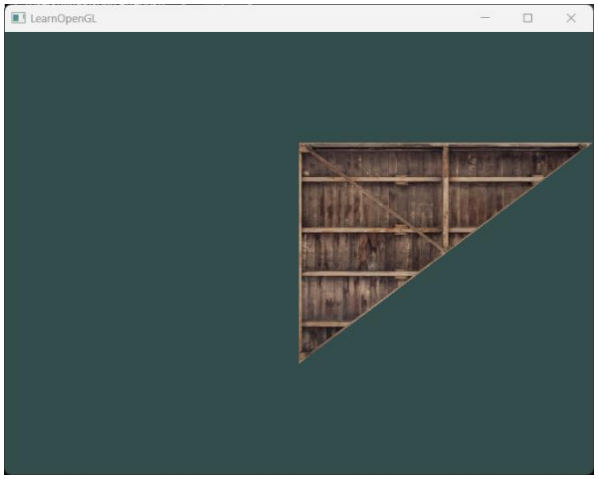
```
glm::mat4 model= glm::mat4(1.0f);
```

```
model= glm::rotate(model, glm::radians(-90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
```



左视图（绕 y 轴逆时针旋转 90 度）:

```
glm::mat4 transform = glm::mat4(1.0f);
transform = glm::rotate(transform, glm::radians(-90.0f), glm::vec3(0.0f, 1.0f, 0.0f));
```

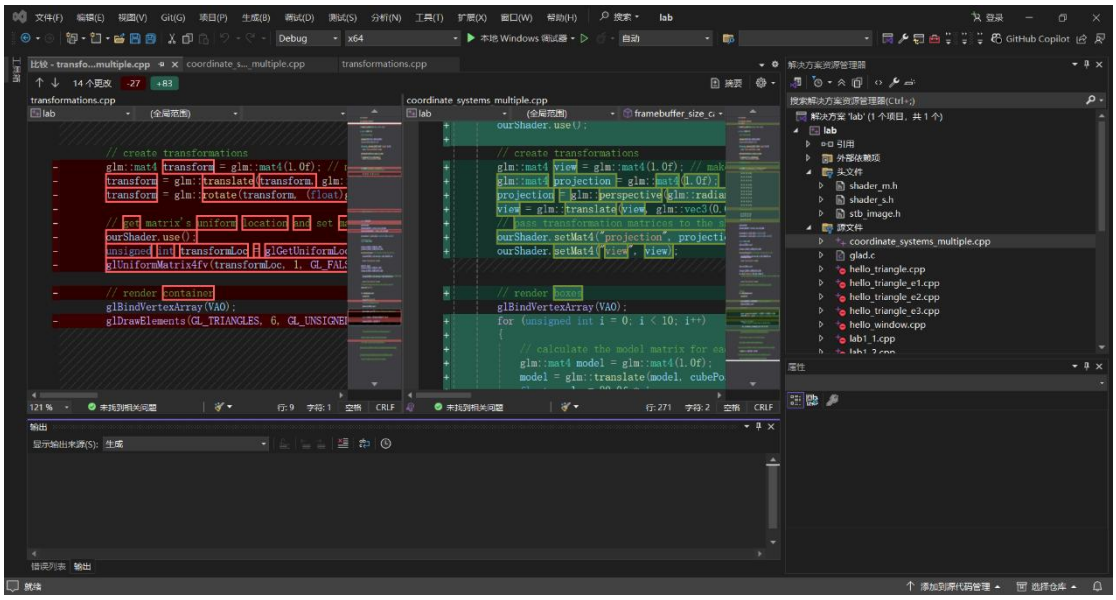


我们可以根据 vertices 与 indices 手动画出构建的立方体，并与绘制的三视图对比是否符合结果。

4. 阅读坐标系统一节，运行示例程序“coordinate_systems_multiple.cpp”

我们也可以先尝试运行 learnopengl 教程前文的“coordinate_systems.cpp”和“coordinate_systems_depth.cpp”，它们的改动并不多。

将“coordinate_systems_multiple.cpp”与”transformations.cpp”对比，主要有以下区别：更改了 vertices，删去 indices 数组及关联的 EBO，增加了每个立方体的位置数组，相对应的更改了绘制方法；增加了深度测试设置，包括更改 glCLear 设置；更改了变换矩阵，增加了透视投影与多个立方体的变换方法；修改了着色器。



5. 为第 3 步的立方体设置透视投影。

根据教程与代码分析，设置透视投影需要处理变换矩阵及相应的着色器。

着色器代码，我们在着色器内基于 transform 矩阵进行物体变换；此处也可以不增加 transform 变量，直接使用 glm 修改 model，效果不变：

```
const char* vertexShaderSource = "#version 330 core\n"
"layout (location = 0) in vec3 aPos;\n"
"layout (location = 1) in vec3 aColor;\n"
"layout (location = 2) in vec2 aTexCoord;\n"
"out vec3 ourColor;\n"
"out vec2 TexCoord;\n"
"uniform mat4 transform;\n"
"uniform mat4 model;\n"
"uniform mat4 view;\n"
"uniform mat4 projection;\n"

"void main()\n"
"{\n"
"   gl_Position = projection * view * model * transform * vec4(aPos, 1.0f);\n"
"   ourColor = aColor;\n"
"   TexCoord = vec2(aTexCoord.x, 1.0 - aTexCoord.y);\n"
"}\0";
```

立方体的 vertices 数组，注意投影矩阵交换了坐标系的左右手性，需要在此处对应修改：

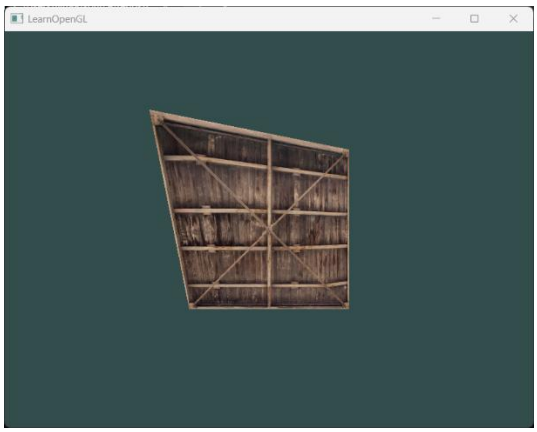
```
    float vertices[] = {
         0.5f,  0.5f, 0.0f, 0.0f, 0.0f, 1.0f, 1.0f, 1.0f,   // top right, blue
         0.5f, -0.5f, 0.0f, 0.0f, 1.0f, 0.0f, 1.0f, 0.0f,// bottom right, green
        -0.5f, -0.5f, 0.0f, 1.0f, 0.0f, 0.0f, 0.0f, 0.0f,// bottom left, red
        -0.5f,  0.5f, 1.0f, 0.0f, 0.0f, 0.0f, 0.0f, 1.0f // top left -> front
    };
```

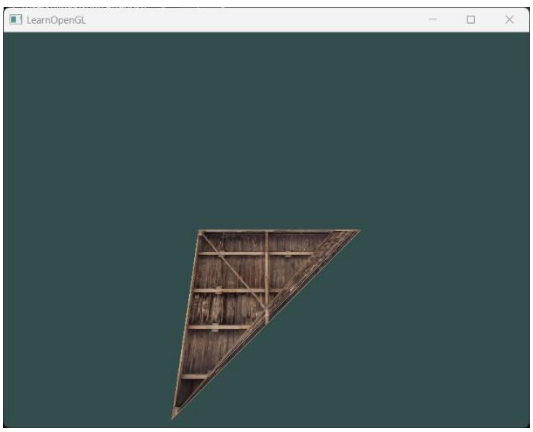
变换矩阵，注意投影矩阵交换了坐标系的左右手性，glm 的旋转方向发生了变换，需要在此处对应修改，即将第 3 步中旋转方向-90 度改为 90 度：

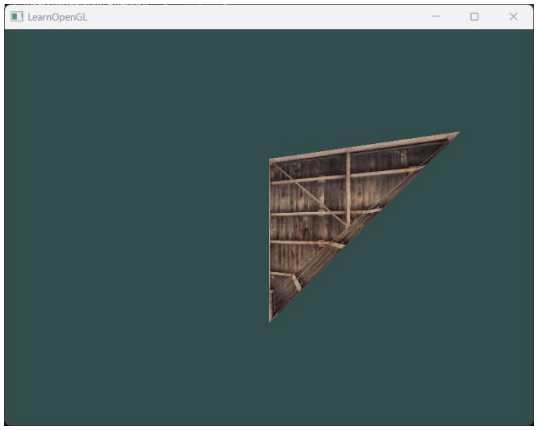
```
        glm::mat4 view = glm::mat4(1.0f); // make sure to initialize matrix to identity matrix first
        glm::mat4 projection = glm::mat4(1.0f);
        projection = glm::perspective(glm::radians(45.0f), (float)SCR_WIDTH / (float)SCR_HEIGHT, 0.1f, 100.0f);
        view = glm::translate(view, glm::vec3(0.0f, 0.0f, -3.0f));
        // pass transformation matrices to the shader

        // calculate the model matrix for each object and pass it to shader before drawing
        glm::mat4 model = glm::mat4(1.0f);
        model = glm::translate(model, glm::vec3(0.0f, 0.0f, 0.0f));

        // create transformations
        glm::mat4 transform = glm::mat4(1.0f); // make sure to initialize matrix to identity matrix first
        transform = glm::rotate(transform, glm::radians(-90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
```

```
        unsigned int transformLoc = glGetUniformLocation(shaderProgram, "transform");
        glUniformMatrix4fv(transformLoc, 1, GL_FALSE, glm::value_ptr(transform));
        unsigned int modelLoc = glGetUniformLocation(shaderProgram, "model");
        glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
        unsigned int projectionLoc = glGetUniformLocation(shaderProgram, "projection");
        glUniformMatrix4fv(projectionLoc, 1, GL_FALSE, glm::value_ptr(projection));
        unsigned int viewLoc = glGetUniformLocation(shaderProgram, "view");
        glUniformMatrix4fv(viewLoc, 1, GL_FALSE, glm::value_ptr(view));
```

绘制三视图，分别得到如下图像。注意在实验一中我们设置最后绘制接近屏幕的两个三角面，覆盖之前的三角面，以校准正视图；此时的俯视图和左视图也是这两个三角面最后绘制，这会导致这两个视角的绘制顺序错误。







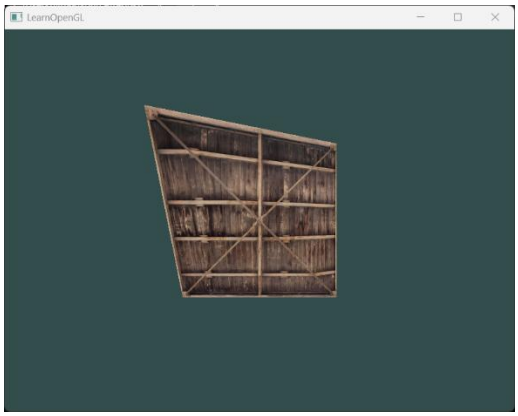
6. 为第 3 步的立方体设置深度测试。

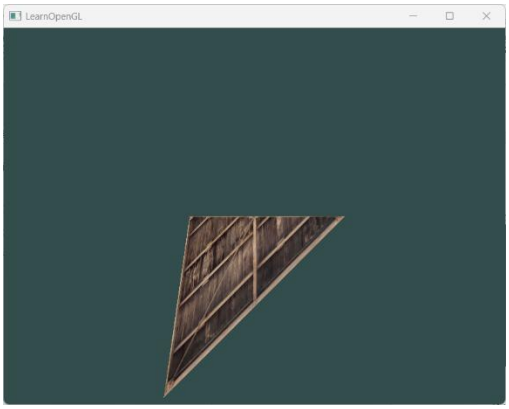
深度测试设置:

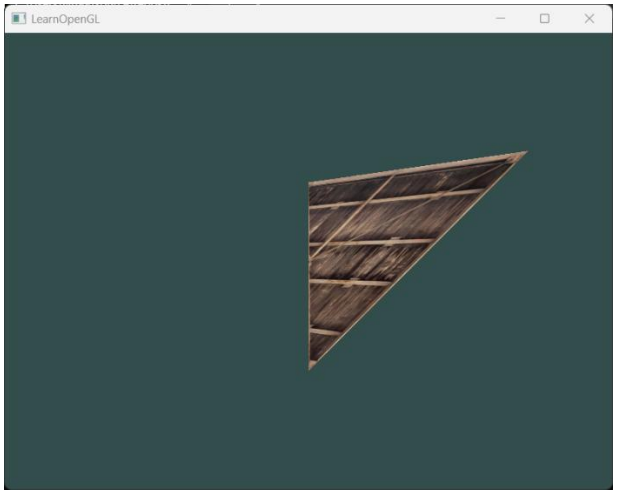
```
glEnable(GL_DEPTH_TEST);
```

```
glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
```

绘制三视图，分别得到以下图像，可以看到纹理变化，正视图纹理不变，俯视图和左视图进行了校准:







7. 立体观察

也可以通过设置持续旋转，查看更立体的效果，并观察设置旋转正负、不同旋转轴的旋转方向变化：

```
    glm::mat4 model = glm::mat4(1.0f);
    model = glm::rotate(model, (float)glfwGetTime() * glm::radians(50.0f), glm::vec3(0.0f, 1.0f, 0.0f));
```